

An autonomous Thales-Pandrosion engine for the geometric extraction of polynomial roots

Mathematical and experimental study of script 113

Ivan Besevic

April 29, 2026

Abstract

We describe the engine `113_pandrosion_autonomous_heuristic_thales_engine.py`, an autonomous extractor of complex roots for dense polynomial systems. The engine does not follow every Bezout path. Instead, it builds a single geometric flow combining a Thales-type homothety, a Riemann-Mobius map, optimization of the starting point, and then local correction in the transformed coordinates. The central idea is to shift the numerical effort from homotopic enumeration to initialization geometry. On the Kostlan systems tested, the engine recovers all requested roots for $ks(2, 12)$ and $ks(2, 14)$, namely 144/144 and 196/196 distinct numerical roots respectively, below the residual threshold used. The document formalizes the flow, clarifies the non-derivative-free nature of 113's local corrector, and reports the reproducible results observed.

1 Problem

Let

$$F = (F_1, \dots, F_n) : \mathbb{C}^n \rightarrow \mathbb{C}^n$$

be a polynomial system. For a generic dense system in which each equation has total degree d , the Bezout number is d^n . A complete homotopy strategy typically associates one path with each expected solution. For cases such as $ks(6, 16)$, this gives

$$16^6 = 16\,777\,216$$

theoretical paths. Engine 113 takes another viewpoint: extract useful and distinct roots by direct geometric exploration, without exhaustive path enumeration.

2 Kostlan systems used

The tests employ a dense Kostlan-type family. For n variables and degree d , one considers monomials

$$z^\alpha = z_1^{\alpha_1} \cdots z_n^{\alpha_n}, \quad |\alpha| \leq d,$$

and random complex coefficients weighted by multinomial factors. In the script's outputs, the number of terms per polynomial is

$$\binom{n+d}{n}.$$

For example, $ks(2, 14)$ contains 120 terms per polynomial, hence 240 terms total for two equations.

3 Single geometric flow

Engine 113 generates an initial point not directly in the original space z , but in an auxiliary coordinate u . For each coordinate, it applies a Mobius transformation coupled with a homothety:

$$y_j = \Lambda p_j \frac{\cos(\theta_j)(u_j/p_j) + \sin(\theta_j)}{-\sin(\theta_j)(u_j/p_j) + \cos(\theta_j)}. \quad (1)$$

Then,

$$z = Ay, \quad (2)$$

where A is a diagonal or quasi-diagonal linear transformation built inside the autonomous engine.

This formula contains several regimes without creating a separate algorithmic branch:

$$\begin{aligned} \theta_j = 0 &\Rightarrow \text{affine homothetic map,} \\ \theta_j \approx \pi/2 &\Rightarrow \text{inversion regime / map toward infinity,} \\ \Lambda \gg 1 &\Rightarrow \text{strong Thales-type homothety.} \end{aligned}$$

The goal is not only to normalize the variables, but to produce points lying in varied numerical basins.

4 Starting-point optimization

The engine applies a *startopt* step. We set

$$G(y) = F(Ay).$$

Starting from the raw point y_0 provided by (1), we test a small set of radial and directional perturbations, then keep the candidate that minimizes a starting energy of the form

$$E(y) = \log(1 + \|G(y)\|) + \varepsilon \log(1 + \|y\|), \quad (3)$$

with ε very small. Geometrically, this step amounts to choosing a better starting point in order to reduce the number of successive proportional constructions needed for correction.

5 Local corrector of 113

It is important to characterize the nature of engine 113 correctly. After the geometric flow, the local corrector solves

$$J_G(y)\Delta = -G(y), \quad y \leftarrow y + \lambda\Delta, \quad (4)$$

where

$$J_G(y) = J_F(Ay)A. \quad (5)$$

The scalar λ is chosen by a damped proportional search.

Remark 1. *Engine 113 is therefore not derivative-free. Its global geometry is Pandrosion/Thales, but its local corrector uses a Jacobian. The engine, however, makes no use of a hidden root solver: it uses neither `numpy.roots`, nor `scipy`, nor `sympy`, nor any external homotopy. The linear solver `np.linalg.solve` is used solely to solve (4).*

6 Heuristic mechanism

Engine 113 deliberately stays simple. At each trial, it picks parameters

$$(\Lambda, \theta, p, r)$$

from a deterministic grid and a heuristic pressure tied to duplications and failures. When duplications appear, the homothetic power is pushed toward other scales. This rule can be seen as a discrete version of a basin-of-attraction exploration:

$$\text{frequent duplicates} \Rightarrow \text{change scale or map.}$$

The strategy is unique: it is not a collection of competing policies.

7 Algorithm

Input: a system $F : \mathbb{C}^n \rightarrow \mathbb{C}^n$, a target count N , a maximum number of trials P .

Output: a set R of distinct numerical roots.

1. Build the dense system and the evaluator F, J_F .
2. Initialize $R = \emptyset$.
3. For $t = 0, \dots, P - 1$:
 - (a) generate a direction u_t , a homothety Λ_t , an angle θ_t , and a pole p_t ;
 - (b) apply the flow (1)-(2);
 - (c) optimize the starting point via (3);
 - (d) correct via (4);
 - (e) validate the residual $\|F(z)\|$ and add the root if it is distinct.
4. Stop when $|R| = N$ or when the trial budget is exhausted.

8 Experimental results

The following tables reproduce the JSON outputs generated by engine 113. The times are those of the local interactive environment and do not constitute a normalized hardware benchmark.

Case	Bezout	Roots requested	Unique roots	Trials	Duplicates	Time (s)
$ks(2, 12)$	144	144	144	393	132	2.422
$ks(2, 14)$	196	196	196	455	107	2.988
$ks(6, 16)$	16,777,216	1	1	1	0	0.797

Table 1: Main results of engine 113.

The engine was also tested on full or partial multivariate cases. The reported observations are:

$ks(3, 4)$: 64/64 roots,
 $ks(4, 3)$: 81/81 roots,
 $ks(3, 6)$: 80/80 requested roots,
 $ks(4, 4)$: 80/80 requested roots,
 $ks(5, 3)$: 50/50 requested roots,
 $ks(6, 4)$: 10/10 requested roots.

These tests confirm that the code is not specialized to the bivariate case.

Case	Terms per polynomial	Total terms	Max residual	Mean residual
$ks(2, 12)$	91	182	$9.91 \cdot 10^{-9}$	$1.81 \cdot 10^{-9}$
$ks(2, 14)$	120	240	$9.96 \cdot 10^{-9}$	$1.79 \cdot 10^{-9}$
$ks(6, 16)$	74,613	447,678	$1.40 \cdot 10^{-11}$	$1.40 \cdot 10^{-11}$

Table 2: Numerical residuals of accepted roots in the 113 tests.

9 Discussion

The main interest of 113 is to show that strong geometrization of the initialization can replace, for the extraction of many roots, the systematic tracking of Bezout paths. The success on $ks(2, 12)$ and $ks(2, 14)$ is particularly noteworthy because the algorithm does not explicitly construct the d^n starting paths. On the other hand, 113 remains locally an engine with derivatives. To obtain a version more faithful to the pure Pandrosion idea, one must replace J_G with an exact slope matrix $Q(a, b)$. This is precisely the subject of engine 115.

10 Reproducibility

Commands used for the main cases:

```
python -S 113_pandrosion_autonomous_heuristic_thales_engine.py \
  --cases 2,12 --equation-normalize --count 144 --pool 5000 \
  --epochs 24 --accept 1e-8 --cluster-sep 1e-8 \
  --startopt-candidates 12
```

```
python -S 113_pandrosion_autonomous_heuristic_thales_engine.py \
  --cases 2,14 --equation-normalize --count 196 --pool 7000 \
  --epochs 24 --accept 1e-8 --cluster-sep 1e-8 \
  --startopt-candidates 12
```

11 Conclusion

Engine 113 proposes an autonomous, multivariate scheme with no dependency on older scripts. Its contribution is the single geometric flow combining strong homothety, Riemann-Mobius maps, and starting-point optimization. This global layer makes the local correction very efficient on the Kostlan benchmarks tested. Its mathematical limitation is clear: the local corrector is of Jacobian type. This limitation naturally motivates a pure Pandrosion version based on an exact slope matrix.